Docker: fundamentos y su importancia en la Arquitectura de Software

Arquitectura de Software

Presentado por:

Christian Javier Benitez Castro - T00082581

Docente:

Luis Giovanny Carreño Ortiz

Universidad Tecnológica de Bolívar

Cartagena de Indias

16/10/26

Introducción

Actualmente, desarrollar una aplicación no consiste solamente en escribir el código: también hay que lograr que funcione en distintos equipos, con las versiones y configuraciones adecuadas. Docker ayuda a resolver esta dificultad al permitir que una aplicación se ejecute dentro de un entorno preparado de antemano. En Arquitectura de Software resulta especialmente útil cuando un sistema tiene varios componentes, como una API, una base de datos y otros servicios que deben comunicarse. En este informe se presentan sus conceptos, herramientas principales, comandos y su relación con el diseño de sistemas.

Desarrollo

1. ¿Qué es Docker?

Docker es una plataforma para crear, distribuir y ejecutar aplicaciones mediante contenedores. Un contenedor reúne el programa y los elementos que necesita, como bibliotecas y configuraciones, para reducir las diferencias entre el computador del desarrollador y el lugar donde se ejecutará. Por ejemplo, una API escrita en Go puede ejecutarse en otra máquina sin exigir que la persona instale manualmente todas sus dependencias.

A grandes rasgos, un contenedor es un proceso aislado que comparte el *kernel* del sistema donde funciona el motor de contenedores. En Linux se aprovechan mecanismos de aislamiento y control de recursos. En Windows, Docker Desktop normalmente utiliza un entorno Linux mediante WSL 2 para ejecutar contenedores Linux. Por eso un contenedor es más liviano que una máquina virtual: esta última necesita su propio sistema operativo y su kernel, mientras que varios contenedores pueden utilizar el mismo.

2. ¿Cuáles son sus componentes?

- **CLI (cliente Docker):** es la interfaz donde se escriben órdenes como *docker ps* o *docker run*. Envía las solicitudes al motor.
- **Daemon (dockerd):** es el proceso que recibe las órdenes, construye imágenes y administra contenedores, redes y almacenamiento.
- **Dockerfile:** archivo que contiene instrucciones para construir una imagen, por ejemplo elegir una base, copiar archivos e instalar dependencias.
- **Imagen:** plantilla formada por capas que guarda lo necesario para crear contenedores; por sí sola no es una aplicación ejecutándose.
- **Contenedor:** instancia creada a partir de una imagen, con su propio entorno de ejecución; se puede iniciar, detener o eliminar.
- **Registro:** lugar donde se almacenan y comparten imágenes. Docker Hub es un ejemplo; desde allí se pueden descargar con *pull* o publicar con *push*.

Relación entre los componentes: el usuario envía una orden desde la CLI y el daemon la ejecuta. La cadena principal es **Dockerfile → imagen → contenedor**: *docker build* construye la imagen y *docker run* crea un contenedor. En lugar de construirla, también se puede descargar una imagen disponible en un registro.

3. ¿Cuáles son sus principales funcionalidades?

- **Portabilidad:** permite trasladar una aplicación entre equipos compatibles. Por ejemplo, ejecutar en otro computador la misma imagen de una API en Go.
- **Aislamiento:** separa los procesos y las dependencias de distintos programas. Por ejemplo, utilizar dos versiones de Python en contenedores diferentes.
- **Versionado:** las etiquetas ayudan a identificar imágenes y elegir una versión. Por ejemplo, solicitar *mysql:8.0* para un proyecto de clase.
- **Reproducibilidad:** el equipo puede trabajar con una configuración común. Por ejemplo, levantar API y MySQL con un archivo de Docker Compose.
- **Automatización y escalabilidad:** facilita crear nuevas instancias y repetir las tareas de construcción y prueba. Por ejemplo, construir una imagen en un proceso de integración continua.

4. ¿Cuáles son sus principales comandos?

a) Tabla de comandos básicos

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen.	docker pull ubuntu
docker run	Crea y arranca un contenedor.	docker run --name prueba ubuntu
docker ps	Muestra los contenedores activos.	docker ps
docker ps -a	Muestra también los detenidos.	docker ps -a
docker logs	Consulta la salida del contenedor.	docker logs mi_api
docker exec	Ejecuta órdenes en uno activo.	docker exec -it mi_api sh
docker stop / start	Detiene o inicia un contenedor.	docker stop mi_api
docker rm	Elimina un contenedor.	docker rm mi_api
docker images	Lista las imágenes descargadas.	docker images
docker rmi	Elimina una imagen local.	docker rmi ubuntu
docker build	Construye una imagen propia.	docker build -t mi-api .
docker compose up	Levanta los servicios definidos.	docker compose up -d

b) Tres comandos explicados con más detalle

docker run: además de crear el contenedor, permite ponerle un nombre con **--name**, ejecutarlo en segundo plano con **-d**, publicar puertos con **-p**, definir variables con **-e** y guardar datos mediante volúmenes con **-v**. Lo usaría para iniciar una base de datos de pruebas.

docker exec: ejecuta una orden en un contenedor que ya está funcionando. La combinación **-it** permite una sesión interactiva con terminal. Lo usaría para consultar archivos o revisar la configuración interna.

docker compose up: lee el archivo de configuración y arranca varios servicios relacionados. **--build** vuelve a construir las imágenes necesarias y **-d** deja los servicios en segundo plano. Lo utilizaría al trabajar con una API y MySQL juntos.

5. ¿Qué problema resuelve Docker?

Uno de los problemas clásicos del desarrollo es escuchar: *"en mi máquina funciona"*. A veces una aplicación corre correctamente en el equipo de quien la creó, pero falla en el de un compañero o en el servidor. Esto puede ocurrir porque cada ambiente tiene una versión distinta del lenguaje, dependencias diferentes, configuraciones incompletas o incluso otro sistema operativo. También sucede cuando la instalación se hace manualmente y algún paso se olvida.

Por ejemplo, supongamos que un grupo desarrolla un servicio con Python y MySQL. Antes de usar Docker, cada integrante instala ambos programas por su cuenta: uno utiliza una versión más antigua de Python y otro tiene ocupado el puerto de MySQL. Al preparar imágenes y un archivo Compose, todos pueden levantar servicios con las versiones y configuraciones acordadas. Esto no evita cualquier error, pero sí disminuye considerablemente las diferencias entre entornos.

6. ¿Por qué es importante dentro de una Arquitectura de Software?

Docker ayuda a llevar el diseño de una arquitectura a un entorno que puede ejecutarse de manera consistente. En un sistema compuesto por una API, una base de datos y un intermediario de mensajes, es conveniente administrar cada componente en su propio contenedor. Así es posible configurar, actualizar o reiniciar uno sin tener que empaquetarlos todos dentro de un mismo proceso. Aun así, la comunicación entre servicios y el manejo de sus datos deben diseñarse cuidadosamente.

Otra ventaja es el despliegue: una imagen que se ha probado durante el desarrollo puede utilizarse también en pruebas y producción, cambiando únicamente las configuraciones que correspondan. Además, resulta más sencillo desplegar nuevas instancias de servicios que reciben muchas solicitudes. Esto es una base útil para trabajar en servidores o plataformas de nube, normalmente junto a herramientas de orquestación cuando el sistema crece.

En los **microservicios**, diferentes funciones del sistema pueden construirse y desplegarse como servicios separados. En **SOA**, los contenedores pueden alojar servicios reutilizables que se comunican mediante interfaces definidas. En una **arquitectura orientada a eventos**, componentes como productores, consumidores y un broker de mensajes pueden ejecutarse en contenedores independientes. Incluso una arquitectura por capas, como la de la API de biblioteca del curso, puede empaquetarse con Docker y conectarse a una base de datos en otro contenedor.

Conclusión

Después de investigar Docker, considero que su principal aporte es facilitar que un proyecto se ejecute de forma parecida en distintos computadores, sin depender tanto de instalaciones manuales. También entendí que no reemplaza el diseño de la arquitectura: organizar correctamente los servicios, la información y la comunicación sigue siendo responsabilidad del equipo. Lo usaría especialmente en trabajos universitarios que tengan una API y una base de datos, porque ayudaría a compartir el proyecto, detectar errores de configuración y trabajar con las mismas herramientas.

Referencias

1. Docker Inc. (s. f.). *What is Docker?* Docker Docs. https://docs.docker.com/get-started/docker-overview/

2. Docker Inc. (s. f.). *What is a container?* Docker Docs. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

3. Docker Inc. (s. f.). *What is an image?* Docker Docs. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/

4. Docker Inc. (s. f.). *How Compose works.* Docker Docs. https://docs.docker.com/compose/intro/compose-application-model/

5. Docker Inc. (s. f.). *Docker CLI reference.* Docker Docs. https://docs.docker.com/reference/cli/docker/

6. Carreño, L. G. (2026). *Taller del 2do Corte — Docker.* Material de clase, Arquitectura de Software.

7. Repositorio de la asignatura. (2026). *Biblioteca API — README.* GitHub. https://github.com/carrenoutb/biblioteca-api